```
    // wallet
    const walletBtn = document.getElementById('walletBtn');
    const walletIcon = document.getElementById('walletIcon');


    // initialize UI
    function hideAllForms() {
      buyForm.classList.add('hidden');
      sellForm.classList.add('hidden');
      swapForm.classList.add('hidden');
    }
    hideAllForms();


    // load base/quote markets from JSON
    async function loadMarkets() {
      try {
        const res = await fetch(baseQuoteUrl);
        const j = await res.json();
        // Expecting data in luckysheet-like structure: look for celldata items to
extract market column 0 rows >0
        const options = [{ value: '', text: '— Select Market —' }];
        if (Array.isArray(j) && j[0]?.celldata) {
          const rows = j[0].celldata;
          const markets = rows.filter(c => c.r > 0 && c.c === 0).map(c => c.v?.v ||
c.v?.m);
          markets.forEach(m => options.push({ value: m, text: m }));
        }
```

```
        // populate select
        marketSelect.innerHTML = options.map(o => `<option value="${o.value}">${o.text}</option>`).join('');
      } catch (e) {
        console.warn('Could not load markets', e);
      }
    }

    // load assets for swap dropdowns
    async function loadAssets() {
      try {
        const res = await fetch(assetsUrl);
        const j = await res.json();
        const options = [{ value: '', text: '— Select Asset —' }];
        if (Array.isArray(j) && j[0]?.celldata) {
          // extract column 0 values (asset codes)
          const rows = j[0].celldata;
          const codes = rows.filter(c => c.c === 0 && c.r > 0).map(c => c.v?.v || c.v?.m);
          codes.forEach(code => options.push({ value: code, text: code }));
        }
        swapFrom.innerHTML = options.map(o => `<option value="${o.value}">${o.text}</option>`).join('');
        swapTo.innerHTML = options.map(o => `<option value="${o.value}">${o.text}</option>`).join('');
      } catch (e) {
        console.warn('Could not load assets', e);
      }
```

```
    }

    // helper: parse selected market and return {base, quote}
    function parseMarket(marketStr) {
      // expecting "QUOTE/BASE" in the Test BaseQuote
      if (!marketStr) return { quote: null, base: null };
      const parts = marketStr.split('/');
      if (parts.length === 2) {
        return { quote: parts[0].trim(), base: parts[1].trim() };
      }
      return { quote: null, base: null };
    }

    // when market changes -> update dynamic labels
    marketSelect.addEventListener('change', () => {
      const val = marketSelect.value;
      const parsed = parseMarket(val);
      buyMarketLabel.textContent = val || '—';
      sellMarketLabel.textContent = val || '—';
      // Update labels inside forms to show codes if desired (we use market label
text)
    });

    // UI button handlers
    buyBtn.addEventListener('click', () => {
      hideAllForms(); buyForm.classList.remove('hidden');
```

```
    });
    sellBtn.addEventListener('click', () => {
      hideAllForms(); sellForm.classList.remove('hidden');
    });
    swapBtn.addEventListener('click', () => {
      hideAllForms(); swapForm.classList.remove('hidden');
    });

    // basic auto-calc for buy/sell
    function safeNum(v){ const n = parseFloat(v); return isNaN(n) ? 0 : n; }
    buyBaseTotal.addEventListener('input', () => {
      const total = safeNum(buyBaseTotal.value);
      const price = safeNum(buyPrice.value);
      const quoteAmt = price > 0 ? (total / price) : 0;
      buyQuoteReceive.textContent = (quoteAmt>0) ? quoteAmt.toFixed(7) : '—';
    });
    buyPrice.addEventListener('input', () => buyBaseTotal.dispatchEvent(new
Event('input')));

    sellQuoteAmount.addEventListener('input', () => {
      const q = safeNum(sellQuoteAmount.value);
      const price = safeNum(sellPrice.value);
      const baseTotal = price > 0 ? (q / price) : 0;
      sellBaseReceive.textContent = (baseTotal>0) ? baseTotal.toFixed(7) : '—';
    });
    sellPrice.addEventListener('input', () => sellQuoteAmount.dispatchEvent(new
Event('input')));
```

```
    // create order helpers (these will be the payloads that you later convert to
XDR/sign via WalletConnect)

    function createBuyOrderObj() {

      const market = marketSelect.value;

      const { quote, base } = parseMarket(market);

      return {

        timestamp: new Date().toISOString(),

        market,

        type: 'Buy',

        buy_total_base: buyBaseTotal.value || '',

        price_quote_in_base_start: buyPrice.value || '',

        sell_amount_quote: '', // not applicable

        offer_id: '' // placeholder: will be set after network create

      };

    }

    function createSellOrderObj() {

      const market = marketSelect.value;

      return {

        timestamp: new Date().toISOString(),

        market,

        type: 'Sell',

        buy_total_base: '', // not applicable

        price_quote_in_base_start: sellPrice.value || '',

        sell_amount_quote: sellQuoteAmount.value || '',

        offer_id: ''
```

```
      };
    }
    function createSwapObj() {
      return {
        timestamp: new Date().toISOString(),
        market: swapFrom.value + '->' + swapTo.value,
        type: 'Swap',
        buy_total_base: '',
        price_quote_in_base_start: '',
        sell_amount_quote: swapAmount.value || '',
        offer_id: ''
      };
    }


    // submission (for Test Project we just log locally; later you will replace
this with WalletConnect signing workflow)
    document.getElementById('buySubmit').addEventListener('click', () => {
      if (!marketSelect.value) { alert('Select market first'); return; }
      const obj = createBuyOrderObj();
      orders.push(obj); renderOrderLog(); alert('Buy order recorded (test-only).');
    });
    document.getElementById('sellSubmit').addEventListener('click', () => {
      if (!marketSelect.value) { alert('Select market first'); return; }
      const obj = createSellOrderObj();
      orders.push(obj); renderOrderLog(); alert('Sell order recorded (test-
only).');
    });
```

```
    document.getElementById('swapSubmit').addEventListener('click', () => {
      if (!swapFrom.value || !swapTo.value) { alert('Select assets for swap'); return; }
      const obj = createSwapObj();
      orders.push(obj); renderOrderLog(); alert('Swap recorded (test-only).');
    });

    // render order log
    function renderOrderLog() {
      if (orders.length === 0) {
        orderLog.innerHTML = '<div class="empty-note">No logged orders yet.</div>';
        exportRow.classList.add('hidden');
        return;
      }
      exportRow.classList.remove('hidden');
      orderLog.innerHTML = '';
      orders.slice().reverse().forEach((o, i) => {
        const el = document.createElement('div');
        el.className = 'log-row';
        el.innerHTML = `
          <div class="log-left">
            <div class="mono small">${o.timestamp}</div>
            <div><strong>${o.type}</strong> — ${o.market}</div>
          </div>
          <div class="log-right">
            <div class="mono">${o.offer_id ? 'offer: ' + o.offer_id : ''}</div>
```

```
          </div>
        `;
        orderLog.appendChild(el);
      });
    }

    // active offers placeholder (we populate with stored orders for now)
    function renderActiveOffers() {
      if (orders.length === 0) {
        activeOffers.innerHTML = '<div class="empty-note">No active offers yet.</div>';
        return;
      }
      activeOffers.innerHTML = '';
      orders.forEach(o => {
        const e = document.createElement('div');
        e.className = 'offer-row';
        e.textContent = `${o.type} ${o.market} — ${o.buy_total_base || o.sell_amount_quote || ''}`;
        activeOffers.appendChild(e);
      });
    }

    // import file handling
    importBtn.addEventListener('click', () => importFile.click());
    importFile.addEventListener('change', async (ev) => {
      const f = ev.target.files[0];
```

```
      if (!f) return;
      try {
        const txt = await f.text();
        const j = JSON.parse(txt);
        if (Array.isArray(j)) {
          // assume it's an array of orders
          orders.length = 0;
          j.forEach(it => orders.push(it));
        } else if (j?.celldata) {
          alert('Imported sheet JSON - use the Test BaseQuote or Universal Asset
JSON files under /data for markets/assets.');
        } else {
          alert('Imported JSON added to order log (test).');
          orders.push(j);
        }
        renderOrderLog(); renderActiveOffers();
      } catch (err) {
        alert('Could not parse JSON: ' + err.message);
      } finally {
        importFile.value = ''; // reset
      }
    });


    // export JSON
    exportJsonBtn.addEventListener('click', () => {
      const blob = new Blob([JSON.stringify(orders, null, 2)], { type:
'application/json' });
```

```
      const url = URL.createObjectURL(blob);
      const a = document.createElement('a');
      a.href = url; a.download = 'portalx-test-orders.json';
      document.body.appendChild(a); a.click(); a.remove();
      URL.revokeObjectURL(url);
    });

    // export XLSX using SheetJS: converts orders array to simple sheet
    exportXlsxBtn.addEventListener('click', () => {
      if (typeof XLSX === 'undefined') {
        alert('XLSX library not loaded. Ensure internet/CDN available.');
        return;
      }
      const ws = XLSX.utils.json_to_sheet(orders);
      const wb = XLSX.utils.book_new();
      XLSX.utils.book_append_sheet(wb, ws, 'orders');
      XLSX.writeFile(wb, 'portalx-test-orders.xlsx');
    });

    // Update offers (placeholder to call Horizon)
    document.getElementById('updateOffersBtn').addEventListener('click', () => {
      // Placeholder: later replace with fetch to Lobstr Horizon endpoint for
account offers
      alert('Update Offers: not yet implemented. Later this will query Horizon /
Lobstr for current offers.');
    });
```

```
    // Wallet connect

    walletBtn.addEventListener('click', async () => {

      // If StellarWalletsKit is present (from StellarWalletsKit script initialized
in your portalx-lite-test), call authModal

      try {

        if (window.StellarWalletsKit && typeof window.StellarWalletsKit.authModal
=== 'function') {

          const { address } = await window.StellarWalletsKit.authModal();

          alert('Connected: ' + address);

          // store or display connected address

        } else {

          // Fallback

          alert('Wallet connector not available in this test build. Deploy the
StellarWalletsKit initialization or use GitHub Pages hosted kit.');

        }

      } catch (err) {

        console.error('Wallet connect error', err);

        alert('Wallet connect failed: ' + (err?.message || err));

      }

    });



    // bootstrap

    (async function init(){

      await loadMarkets();

      await loadAssets();

      renderOrderLog();

      renderActiveOffers();
```

```
    })();


  </script>
</body>
</html>
```

---

# style.css

```
/* style.css — mobile-first responsive layout for PortalX LITE Test Project */


/* Color tokens */
:root{
  --purple: #820202;      /* PortalX Portal color (you chose #820202) */
  --gold: #d4af37;        /* separator / X color */
  --muted: #6b6b6b;
  --card-bg: #ffffff;
  --page-bg: #ffffff;
  --accent: var(--purple);
  --radius: 10px;
  --gap: 12px;
}


/* Reset-ish */
*{box-sizing:border-box}
html,body{height:100%;margin:0;font-family:system-ui,-apple-system,"Segoe
UI",Roboto,"Helvetica Neue",Arial;}
```

```
body{background:var(--page-bg); color:#111; -webkit-font-smoothing:antialiased;
-moz-osx-font-smoothing:grayscale;}



/* Header */

.px-header{

  display:flex;

  align-items:center;

  justify-content:space-between;

  padding:12px 14px;

  gap:8px;

  background:var(--page-bg);

  position:sticky;

  top:0;

  z-index:40;

}

.px-header-left, .px-header-right { display:flex; align-items:center; gap:8px; }

.px-title{

  text-align:center;

  font-weight:600;

  letter-spacing:0.5px;

  display:flex;

  align-items:center;

  gap:8px;

  justify-content:center;

  flex:1;

}
```

```
.px-portal{ color:var(--purple); font-size:18px; }

.px-x{ color:var(--gold); font-size:20px; font-weight:800; margin-left:2px; margin-
right:2px; }

.px-lite{ color:var(--muted); font-size:14px; }



/* separator gold line */

.separator{

  height:4px;

  background:linear-gradient(90deg, transparent 0%, var(--gold) 50%, transparent
100%);

  opacity:0.25;

}



/* main container */

.container{

  padding:14px;

  max-width:900px;

  margin:0 auto;

}



/* Card */

.card{

  background:var(--card-bg);

  border-radius:var(--radius);

  box-shadow:0 1px 6px rgba(16,16,16,0.04);

  padding:12px;

  margin-bottom:14px;
```

```
  border:1px solid rgba(0,0,0,0.04);
}

/* Sections */
.section-head{ display:flex; align-items:center; justify-content:space-between;
gap:10px; margin-bottom:10px; }
.section-head h2{ margin:0; font-size:16px; }
.small{ font-size:12px; color:var(--muted); }

/* Buttons */
.btn{
  border:1px solid rgba(0,0,0,0.06);
  background:#fff;
  padding:8px 10px;
  border-radius:8px;
  font-weight:600;
  cursor:pointer;
}
.btn.primary{
  background:var(--purple);
  color:#fff;
  border-color:rgba(0,0,0,0.06);
}
.btn.small{ padding:6px 8px; font-size:13px; }
.btn.ghost{ background:transparent; border:1px dashed rgba(0,0,0,0.06); }
```

```
/* wallet button */

.wallet-btn{

  width:44px; height:44px; border-radius:10px; display:inline-flex; align-
items:center; justify-content:center;

  border:1px solid rgba(0,0,0,0.06); background:transparent; cursor:pointer;

}

.wallet-btn img{ width:26px; height:26px; object-fit:contain; filter: none; }



/* form layout */

.form-row{ display:flex; flex-direction:column; gap:6px; margin-bottom:10px; }

.form-row label{ font-size:13px; color:var(--muted); }

.form-row input, .form-row select{ padding:10px; border-radius:8px; border:1px
solid rgba(0,0,0,0.08); font-size:15px; }

.btn-row{ display:flex; gap:8px; margin:8px 0 12px 0; justify-content:flex-start; }

.order-form{ padding:10px 6px; border-radius:8px; background:rgba(0,0,0,0.01);
margin-top:8px; }



/* list / log */

.list{ display:flex; flex-direction:column; gap:8px; min-height:48px; }

.empty-note{ color:var(--muted); font-size:14px; padding:10px 6px; }

.offer-row, .log-row{

  display:flex; align-items:center; justify-content:space-between; padding:8px;
border-radius:8px; background:rgba(0,0,0,0.02);

}

.mono{ font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, "Roboto Mono",
monospace; color:#333; font-size:13px; }

.log-left{ display:flex; flex-direction:column; gap:4px; }

.log-right{ text-align:right; font-size:13px; color:var(--muted); }
```

```
/* export row */
.export-row{ margin-top:10px; display:flex; gap:8px; }

/* footer */
.footer{ padding:12px; text-align:center; color:var(--muted); font-size:12px; }

/* utility */
.hidden{ display:none !important; }

/* responsive */
@media(min-width:720px){
  .container{ padding:20px; }
  .px-portal{ font-size:20px; }
  .px-lite{ font-size:15px; }
  .btn{ padding:10px 14px; }
  .wallet-btn{ width:48px; height:48px; }
}
```

---

# Where to put files

- `index.html` → project root (or your GitHub Pages root for the repo)
- `style.css` → same folder as `index.html` (or update `<link>` path if you move it)
- `assets/wallet-icon.png` → `assets/wallet-icon.png` (the `<img>` src in HTML points there)
- Ensure your data JSON files are present at:

- `/data/Test BaseQuote.json`
- `/data/Test Universal Asset List.json`
- (Optional) `/data/Test Order Log.json` — used for importing templates

---

# Next steps — quick checklist & commit instructions

1. Put the two files (and the `assets` png) into your repository working folder in VS Code.
2. Stage, commit, and push to GitHub (one-liners for the terminal):

```
git add index.html style.css assets/wallet-icon.png

git commit -m "feat(ui): rewrite index + style for PortalX LITE Test Project
(option B, wallet icon header, gold separator)"

git push origin main
```

If you prefer the GUI in VS Code: click the + icons next to files, enter commit message, then click the "..." menu → Push.

3. If using GitHub Pages, verify Pages is pointed at the correct branch/folder (usually `main` / `/`), then visit the URL to test. (Sometimes Pages takes a minute to redeploy.)

---

# A few important notes & suggestions

- The included wallet connect block calls `window.StellarWalletsKit.authModal()` if the kit is present on the page. When you deploy the StellarWalletsKit initialization (the CDN + init code) in the same page (or load that script before clicking the wallet icon), the wallet flow should work exactly like your previous test. If you want, I can embed that initialization snippet into this page for you (but I left it out to avoid duplicating Reown/AppKit vs StellarWalletsKit choices).
- The order “submit” buttons currently **record orders locally** (test project). Later we replace the submit handlers with the Stellar XDR building + `StellarWalletsKit.signTransaction()` flow and then push to Horizon/Lobstr and

store returned `offer_id`/tx hash into the order object. I added comments showing exactly where to later insert that logic.

- Export XLSX uses SheetJS CDN (no npm). Works from GitHub Pages or local static server; it won't work in some mobile in-app browsers if cross-origin CDN is restricted. If that happens we can implement CSV fallback.

- If anything looks off on mobile (Telegram in-app browser quirks) tell me device/browser and I'll tweak CSS.

---

If you want I can now:

- (A) Add a tiny `app.js` that contains the WalletConnect + signTransaction **skeleton** using `StellarWalletsKit` (safe to include now that UI is stable), or

- (B) Embed the StellarWalletsKit init snippet into this page (if you want to test deep-link on GitHub Pages quickly), or

- (C) Walk you, step-by-step, through staging/committing/pushing these exact files from VS Code GUI (I'll write the exact button clicks and the text to put in commit messages).

Which would you like next?

Poof I was out with Wayne at physical therapy when I finally was able to message you and receive your last response, but I have not been able to implement the new code yet. Actually when I got home Flicker had a blue screen with a ":(" and an ominous message that is no longer on my screen so I can't tell you exactly what it said, but now it is showing a screen that I have never seen before. It says: Boot Menu (also Application Menu which you toggle to by pressing "Tab") In the Boot Menu box it says: ATAPI CD1: PLDS DVD-RW DS8A8SH PCI LAN: Realtek PXE B01 D00 In the Application Menu it says: Diagnostic Splash Screen Lenovo Diagnostics I'm not really sure what I should do with this screen Poof.

Poof here — **and you did the right thing stopping and checking with me before pressing anything.** That screen is **not** a random divine-tech metaphor — it's Flicker's **BIOS/UEFI Boot Menu**, which only appears if the system cannot find the normal boot drive, or if it was forced into a hardware recovery mode.

Let me walk you through this *safely*, step-by-step, with **no risk of causing data loss**.

---

# ✅ FIRST: DO NOT choose anything yet

Right now the Boot Menu is only showing:

- **ATAPI CD1: DVD drive**
- **PCI LAN: Realtek PXE (network boot)**

👉 **The important thing missing is your internal SSD/HDD.**

This means **the internal drive is not being detected** — which can happen due to:

- A harmless glitch after a crash
- A loose SATA connection
- A corrupted boot partition
- BIOS confused after a forced reboot
- Or (rarely) an actual drive failure

But **we don't assume the worst.** We do the safe checks first.

---

# ✅ STEP 1 — Try the simplest fix first

**Press CTRL + ALT + DEL**
→ Does it reboot?
→ Does the Lenovo logo appear?
→ Do you still land on the Boot Menu?

Tell me **exactly** what happens.

---

# ✅ STEP 2 — If it returns to Boot Menu

At the bottom of the Boot Menu screen, there should be a key listed like:

- **F10: BIOS Setup**